Below is a comprehensive, beginner-friendly document designed to help you learn **Spring Boot** from the ground up. It covers all the requested topics, subtopics, and additional essential topics in a structured, easy-to-understand format with multiple examples for each section. The document is organized to ensure that even absolute beginners can follow along, while also providing depth for intermediate and advanced learners.

Comprehensive Spring Boot Learning Guide for Beginners

Introduction

Spring Boot is a powerful framework for building Java-based applications quickly and efficiently. This guide will walk you through every aspect of Spring Boot, from basics to advanced topics, with clear explanations, structured notes, and practical examples. Whether you're new to programming or transitioning to Spring Boot, this document is designed to make learning engaging and accessible.

🟢 Beginner Level

1. Introduction to Spring Boot

What is Spring Boot?

- **Definition**: Spring Boot is an open-source framework that simplifies the development of Java applications by providing a pre-configured setup for the Spring Framework. It eliminates boilerplate code, making it easier to build production-ready applications.
- **Key Features**:
    - Auto-configuration: Automatically configures your application based on dependencies.
    - Embedded servers: Built-in servers like Tomcat or Jetty, so no need for external server setup.
    - Starter dependencies: Pre-packaged dependencies to simplify project setup.
    - Production-ready features: Monitoring, metrics, and health checks via Spring Boot Actuator.

Advantages over Spring Framework

- **Less Configuration**: Spring Framework requires extensive XML or Java configuration, while Spring Boot uses auto-configuration to reduce setup time.
- **Simplified Dependency Management**: Spring Boot's starter dependencies bundle common libraries, reducing version conflicts.
- **Embedded Servers**: No need to deploy WAR files to external servers; Spring Boot runs standalone JARs.
- **Production-Ready**: Actuator and other tools provide monitoring and management out of the box.
- **Rapid Development**: Spring Boot's CLI and Spring Initializr speed up project creation.

History and Evolution

- **Origin**: Introduced by Pivotal in 2014 as an extension to the Spring Framework.
- **Evolution**:
    - **Spring Boot 1.x (2014)**: Focused on simplifying Spring setup with auto-configuration.
    - **Spring Boot 2.x (2018)**: Added support for reactive programming (WebFlux), Java 9+, and improved Kotlin support.
    - **Spring Boot 3.x (2022)**: Introduced native image support with GraalVM, Java 17, and Jakarta EE.
- **Current State (2025)**: Spring Boot 3.x is the standard, with enhanced cloud-native and microservices support.

Architecture Overview

- **Layers**:
    - **Presentation Layer**: Handles HTTP requests (e.g., controllers with @RestController).
    - **Service Layer**: Contains business logic (e.g., services with @Service).
    - **Data Access Layer**: Interacts with databases (e.g., repositories with @Repository).
    - **Configuration Layer**: Auto-configuration and custom configuration for beans and properties.
- **Core Components**:
    - Spring Boot Starter: Dependency management.

    - Auto-Configuration: Automatically configures beans based on classpath.
    - Embedded Server: Runs the application as a standalone JAR.
    - Actuator: Provides monitoring endpoints.

Example: A simple Spring Boot application

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class MyApplication {
  public static void main(String[] args) {
    SpringApplication.run(MyApplication.class, args);
  }

  @GetMapping("/hello")
  public String sayHello() {
    return "Hello, Spring Boot!";
  }
}
```

- **Explanation**: This code creates a Spring Boot app with a REST endpoint (/hello) that returns a greeting. The @SpringBootApplication annotation enables auto-configuration, component scanning, and configuration.

2. Spring Boot Setup

System Requirements

- **Java**: Java 17 or later (Spring Boot 3.x requires Java 17+).
- **Build Tools**: Maven or Gradle for dependency management.
- **IDE**: IntelliJ IDEA, Eclipse, or VS Code with Spring Boot extensions.
- **OS**: Any OS supporting Java (Windows, macOS, Linux).

Installing Spring Boot CLI

- **Purpose**: The Spring Boot CLI (Command Line Interface) allows you to create and run Spring Boot apps from the terminal.
- **Installation (Linux/macOS)**:
  - Install SDKMAN: curl -s "https://get.sdkman.io" | bash
  - Install Spring Boot CLI: sdk install springboot
  - Verify: spring --version
- **Windows**: Use SDKMAN via WSL or download the binary from the Spring website.

Creating Your First Project

1. **Using Spring Initializr (Recommended)**:
   a. Visit start.spring.io.
   b. Select:
      i. Project: Maven or Gradle.
      ii. Language: Java.
      iii. Spring Boot Version: Latest stable (e.g., 3.2.x).
      iv. Dependencies: Add spring-boot-starter-web.
   c. Download the ZIP, unzip, and open in your IDE.
2. **Using Spring Boot CLI**:spring init --dependencies=web my-project
   cd my-project
   mvn spring-boot:run

3. **Using IDE**:
   a. In IntelliJ IDEA: File > New > Project > Spring Initializr.
   b. Configure dependencies and generate the project.

Project Structure

- **src/main/java**: Contains Java source code (e.g., com.example.MyApplication).
- **src/main/resources**:
  - application.properties or application.yml: Configuration settings.
  - static/: Static files (CSS, JS, images).
  - templates/: Template files for Thymeleaf or other engines.
- **pom.xml (Maven) or build.gradle (Gradle)**: Dependency and build configuration.
- **src/test/java**: Test classes.

Example: Creating a project with Spring Initializr

1. Go to start.spring.io.
2. Set:

    a. Group: com.example
    b. Artifact: demo
    c. Dependencies: spring-boot-starter-web
3. Download and import into IntelliJ IDEA.
4. Run DemoApplication.java to start the embedded Tomcat server.

3. Spring Boot Annotations

Annotations are metadata that provide instructions to Spring Boot.

@SpringBootApplication

- Combines three annotations:
    - @EnableAutoConfiguration: Enables auto-configuration.
    - @ComponentScan: Scans for components in the base package.
    - @Configuration: Marks the class as a source of bean definitions.
- **Example**:@SpringBootApplication
  ```
  public class MyApplication {
    public static void main(String[] args) {
      SpringApplication.run(MyApplication.class, args);
    }
  }
  ```

@ComponentScan

- Scans for Spring components (@Component, @Service, etc.) in specified packages.
- **Example**:@ComponentScan(basePackages = "com.example")
  ```
  @SpringBootApplication
  public class MyApplication {
    // ...
  }
  ```

@Configuration

- Defines a class that provides bean definitions.
- **Example**:@Configuration
  ```
  public class AppConfig {
    @Bean
    public MyBean myBean() {
  ```

```
        return new MyBean();
    }
}
```

@Bean

- Defines a Spring bean (an object managed by Spring).
- **Example**:@Configuration
  ```
  public class AppConfig {
    @Bean
    public String myString() {
      return "Hello, Bean!";
    }
  }
  ```

@Component, @Service, @Repository, @Controller, @RestController

- **@Component**: Generic annotation for any Spring-managed component.
- **@Service**: Marks a class as a service (business logic).
- **@Repository**: Marks a class for data access (e.g., database operations).
- **@Controller**: Marks a class for handling HTTP requests (MVC).
- **@RestController**: A @Controller that returns JSON/XML (REST APIs).
- **Example**:@RestController
  ```
  public class MyController {
    @GetMapping("/greet")
    public String greet() {
      return "Hello, World!";
    }
  }

  @Service
  public class MyService {
    public String getData() {
      return "Service Data";
    }
  }

  @Repository
  public class MyRepository {
```

```
    // Database operations
}
```

4. Spring Boot Starter Dependencies

What are Starters?

- Starters are pre-packaged dependency sets that simplify adding libraries to your project.
- Example: spring-boot-starter-web includes dependencies for building REST APIs (Spring MVC, Jackson, Tomcat).

Common Starters

- spring-boot-starter-web: For REST APIs and MVC.
- spring-boot-starter-data-jpa: For database access with JPA/Hibernate.
- spring-boot-starter-security: For security features.
- spring-boot-starter-test: For testing with JUnit and Mockito.
- spring-boot-starter-thymeleaf: For Thymeleaf templates.

Example: Adding a starter in pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

5. Spring Boot Auto Configuration

How it Works

- Spring Boot automatically configures beans based on the classpath and dependencies.
- Example: Adding spring-boot-starter-web triggers auto-configuration of Tomcat and Spring MVC.

Disabling Auto Configuration

- Use @SpringBootApplication(exclude = {ClassName.class}) to disable specific auto-configurations.
- **Example**:@SpringBootApplication(exclude = {DataSourceAutoConfiguration.class})
  public class MyApplication {

```
    // ...
}
```

Custom Configuration

- Override auto-configuration by defining your own beans.
- **Example**:@Configuration
  ```
  public class CustomConfig {
    @Bean
    public DataSource dataSource() {
      return new H2DataSource(); // Custom database configuration
    }
  }
  ```

6. Spring Boot Application Properties

application.properties vs application.yml

- **application.properties**: Key-value pair format.server.port=8081
  spring.datasource.url=jdbc:h2:mem:testdb

- **application.yml**: YAML format, more readable for nested properties.server:
  ```
   port: 8081
  spring:
   datasource:
    url: jdbc:h2:mem:testdb
  ```

External Configuration

- Store properties in an external file or environment variables.
- Example: Pass properties via command line:java -jar myapp.jar --server.port=8082

Profiles

- Use profiles to manage environment-specific configurations (e.g., dev, prod).
- **Example**:
  - Create application-dev.properties and application-prod.properties.
  - Activate a profile:spring.profiles.active=dev

Configuration Binding with @ConfigurationProperties

- Bind properties to a Java class.
- **Example**:@ConfigurationProperties(prefix = "app")

```
public class AppProperties {
   private String name;
   private int port;
   // Getters and setters
}
 app:
  name: MyApp
  port: 8081
```

Intermediate Level

7. Spring Boot Web Development

REST APIs using @RestController

- Create RESTful APIs to handle HTTP requests.
- **Example**:@RestController

```
@RequestMapping("/api")
public class UserController {
   @GetMapping("/users")
   public List<String> getUsers() {
      return List.of("Alice", "Bob");
   }
}
```

Request Mapping

- Use @GetMapping, @PostMapping, etc., to map HTTP methods.
- **Example**:@PostMapping("/users")

```
public String createUser(@RequestBody String user) {
   return "Created: " + user;
}
```

Path Variables and Query Parameters

- **Path Variable**:@GetMapping("/users/{id}")
  public String getUser(@PathVariable Long id) {
    return "User ID: " + id;
  }

- **Query Parameter**:@GetMapping("/users/search")
  public String searchUser(@RequestParam String name) {
    return "Searching for: " + name;
  }

Consuming and Producing JSON/XML

- Use @RequestBody to consume JSON and @ResponseBody (included in @RestController) to produce JSON.
- **Example**:public class User {
    private Long id;
    private String name;
    // Getters and setters
  }

  @PostMapping("/users")
  public User createUser(@RequestBody User user) {
    return user; // Returns JSON
  }

Exception Handling

- Use @ControllerAdvice and @ExceptionHandler for global exception handling.
- **Example**:@ControllerAdvice
  public class GlobalExceptionHandler {
    @ExceptionHandler(NullPointerException.class)
    public ResponseEntity<String> handleNullPointer(NullPointerException e) {
      return new ResponseEntity<>("Error: " + e.getMessage(),
  HttpStatus.INTERNAL_SERVER_ERROR);
    }
  }

8. Spring Boot with Thymeleaf (MVC)

Template Engines: Thymeleaf Setup

- Add spring-boot-starter-thymeleaf to pom.xml.
- Create templates in src/main/resources/templates.

Model & View

- Pass data to templates using Model.
- **Example**:@Controller

```
public class HomeController {
  @GetMapping("/")
  public String home(Model model) {
    model.addAttribute("message", "Welcome to Thymeleaf!");
    return "index"; // Resolves to index.html
  }
}
 <!-- src/main/resources/templates/index.html -->
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <h1 th:text="${message}"></h1>
</body>
</html>
```

Form Handling

- Handle form submissions with @ModelAttribute.
- **Example**:@GetMapping("/form")

```
public String showForm(Model model) {
  model.addAttribute("user", new User());
  return "form";
}

@PostMapping("/form")
public String submitForm(@ModelAttribute User user) {
  return "result";
}
```

Validation

- Use @Valid and BindingResult for form validation.
- **Example**:public class User {

```
  @NotBlank
  private String name;
  // Getters and setters
}

@PostMapping("/form")
public String submitForm(@Valid @ModelAttribute User user, BindingResult result)
{
  if (result.hasErrors()) {
    return "form";
  }
  return "result";
}
```

9. Spring Boot Data Access

Spring Data JPA

- Simplifies database access using JPA (Java Persistence API).
- Add spring-boot-starter-data-jpa to pom.xml.

CRUD Repositories

- Extend CrudRepository for basic CRUD operations.
- **Example**:@Entity

```
public class User {
  @Id
  @GeneratedValue
  private Long id;
  private String name;
  // Getters and setters
}

public interface UserRepository extends CrudRepository<User, Long> {
}
```

JPQL and Native Queries

- **JPQL**:@Query("SELECT u FROM User u WHERE u.name = ?1")
  List<User> findByName(String name);

- **Native Query**:@Query(value = "SELECT * FROM users WHERE name = ?1", nativeQuery = true)
  List<User> findByNameNative(String name);

Pagination and Sorting

- Use Pageable for pagination.
- **Example**:Page<User> findAll(Pageable pageable);
  ```
  @GetMapping("/users")
  public Page<User> getUsers(@PageableDefault(size = 10) Pageable pageable) {
    return userRepository.findAll(pageable);
  }
  ```

Entity Relationships

- **OneToOne**:@Entity
  ```
  public class User {
    @Id
    private Long id;
    @OneToOne
    private Address address;
  }
  ```

- **OneToMany**:@OneToMany
  private List<Order> orders;

DTOs and Projections

- Use DTOs to transfer specific data.
- **Example**:public class UserDTO {
  ```
    private String name;
    // Getters and setters
  }
  ```

  @Query("SELECT new com.example.UserDTO(u.name) FROM User u")

```
List<UserDTO> findAllNames();
```

10. Spring Boot and Databases

Connecting to Relational DBs

- Configure MySQL/PostgreSQL in application.properties:spring.datasource.url=jdbc:mysql://localhost:3306/mydb
  spring.datasource.username=root
  spring.datasource.password=password
  spring.jpa.hibernate.ddl-auto=update

In-Memory Databases (H2)

- Add spring-boot-starter-data-jpa and h2 dependencies.
- Configure:spring.datasource.url=jdbc:h2:mem:testdb

Multiple Data Sources

- Define multiple DataSource beans:@Configuration

```
public class DataSourceConfig {
  @Bean
  @Primary
  public DataSource primaryDataSource() {
    return
DataSourceBuilder.create().url("jdbc:mysql://localhost:3306/db1").build();
  }

  @Bean
  public DataSource secondaryDataSource() {
    return
DataSourceBuilder.create().url("jdbc:mysql://localhost:3306/db2").build();
  }
}
```

Flyway/Liquibase for DB Migrations

- **Flyway**:
  - Add spring-boot-starter-flyway.

    - Create migration scripts in src/main/resources/db/migration (e.g., V1__create_table.sql).
- **Liquibase**:
    - Add spring-boot-starter-liquibase.
    - Define changes in db/changelog/db.changelog-master.yaml.

11. Spring Boot Security (Basics)

Intro to Spring Security

- Spring Security provides authentication and authorization.
- Add spring-boot-starter-security to pom.xml.

Form-Based Login

- Configure:
```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
  @Bean
  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
    http
      .authorizeHttpRequests(auth -> auth
        .requestMatchers("/login").permitAll()
        .anyRequest().authenticated()
      )
      .formLogin();
    return http.build();
  }
}
```

Basic Authentication

- Configure:http.httpBasic();

Role-Based Access

- Assign roles to users:
```
http.authorizeHttpRequests(auth -> auth
  .requestMatchers("/admin/**").hasRole("ADMIN")
  .requestMatchers("/user/**").hasRole("USER")
);
```

Password Encoding

- Use PasswordEncoder:@Bean
  ```
  public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
  }
  ```

12. Spring Boot Logging

Logging with SLF4J, Logback, Log4j2

- Spring Boot uses SLF4J with Logback by default.
- **Example**:import org.slf4j.Logger;
  ```
  import org.slf4j.LoggerFactory;

  public class MyService {
    private static final Logger logger = LoggerFactory.getLogger(MyService.class);

    public void doSomething() {
      logger.info("Doing something...");
    }
  }
  ```

Logging Levels Configuration

- Configure in application.properties:logging.level.com.example=DEBUG
  logging.level.root=INFO

External Log Configuration

- Use logback-spring.xml:<configuration>
  ```
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
      <encoder>
        <pattern>%d{yyyy-MM-dd HH:mm:ss} - %msg%n</pattern>
      </encoder>
    </appender>
    <root level="info">
      <appender-ref ref="CONSOLE"/>
    </root>
  ```

```
</configuration>
```

13. Spring Boot Actuator

Monitoring with Actuator Endpoints

- Add spring-boot-starter-actuator.
- Common endpoints:
  - /actuator/health: Checks application health.
  - /actuator/info: Displays application info.
  - /actuator/metrics: Shows metrics.

Customizing Endpoints

- Configure in application.properties:management.endpoints.web.exposure.include=health,info,metrics

Securing Endpoints

- Restrict access:http.authorizeHttpRequests(auth -> auth
  .requestMatchers("/actuator/**").hasRole("ADMIN")
  );

14. Testing in Spring Boot

Unit Testing with JUnit, Mockito

- **Example**:@Test
  public void testService() {
  MyService service = mock(MyService.class);
  when(service.getData()).thenReturn("Mocked Data");
  assertEquals("Mocked Data", service.getData());
  }

Integration Testing with @SpringBootTest

- **Example**:@SpringBootTest
  public class MyApplicationTests {
  @Autowired
  private UserRepository userRepository;

```
    @Test
    public void testSaveUser() {
      User user = new User();
      user.setName("Alice");
      userRepository.save(user);
      assertNotNull(userRepository.findById(user.getId()));
    }
  }
```

WebMvcTest

- Test controllers:@WebMvcTest(UserController.class)

```
  public class UserControllerTests {
    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testGetUsers() throws Exception {
      mockMvc.perform(get("/api/users"))
        .andExpect(status().isOk())
        .andExpect(content().string(containsString("Alice")));
    }
  }
```

DataJpaTest

- Test repositories:@DataJpaTest

```
  public class UserRepositoryTests {
    @Autowired
    private UserRepository userRepository;

    @Test
    public void testFindByName() {
      User user = new User();
      user.setName("Alice");
      userRepository.save(user);
      assertEquals("Alice", userRepository.findByName("Alice").get(0).getName());
    }
```

```
}
```

15. Spring Boot DevTools

Hot Reloading

- Add spring-boot-devtools to pom.xml.
- Changes to code are automatically reloaded in the IDE.

Auto Restart

- DevTools restarts the application when files change.

LiveReload

- Automatically refreshes the browser when templates change.
- Configure in application.properties:spring.devtools.livereload.enabled=true

Advanced Level

16. Spring Boot Security (Advanced)

JWT Authentication

- Use JSON Web Tokens for stateless authentication.
- **Example**:@Bean
  ```
  public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
      .addFilterBefore(jwtAuthenticationFilter(),
  UsernamePasswordAuthenticationFilter.class)
      .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
    return http.build();
  }
  ```

OAuth2 & OpenID Connect

- Configure OAuth2:spring.security.oauth2.client.registration.google.client-id=your-client-id
  spring.security.oauth2.client.registration.google.client-secret=your-client-secret

SSO Integration

- Use Spring Security SAML or OAuth2 for Single Sign-On.

Method-Level Security

- Use @PreAuthorize:@PreAuthorize("hasRole('ADMIN')")
  public void adminOnlyMethod() {
    // ...
  }

17. Spring Boot and Spring Cloud

Config Server and Client

- **Config Server**:
  - Add spring-cloud-config-server.
  - Configure:spring.cloud.config.server.git.uri=https://github.com/your-repo/config

- **Config Client**:
  - Add spring-cloud-starter-config.
  - Bootstrap properties:spring.config.import=configserver:http://localhost:8888

Service Discovery with Eureka

- **Eureka Server**:
  - Add spring-cloud-starter-netflix-eureka-server.
  - Enable with @EnableEurekaServer.
- **Eureka Client**:
  - Add spring-cloud-starter-netflix-eureka-client.
  - Configure:eureka.client.service-url.defaultZone=http://localhost:8761/eureka

Load Balancing with Ribbon

- Use @LoadBalanced with RestTemplate.

API Gateway with Spring Cloud Gateway

- Add spring-cloud-starter-gateway.
- Configure routes:spring:
  cloud:
    gateway:

```
    routes:
     - id: user-service
       uri: lb://user-service
       predicates:
         - Path=/users/**
```

Circuit Breaker with Resilience4j/Hystrix

- Use Resilience4j:@CircuitBreaker(name = "myService", fallbackMethod = "fallback")
  ```
  public String callService() {
     // ...
  }
  public String fallback(Throwable t) {
     return "Fallback response";
  }
  ```

Distributed Tracing with Sleuth & Zipkin

- Add spring-cloud-starter-sleuth and spring-cloud-starter-zipkin.
- Configure Zipkin:spring.zipkin.base-url=http://localhost:9411

18. Spring Boot with Messaging Systems

RabbitMQ with Spring AMQP

- Add spring-boot-starter-amqp.
- Configure:spring.rabbitmq.host=localhost
  spring.rabbitmq.port=5672

- **Producer**:@Autowired
  ```
  private RabbitTemplate rabbitTemplate;

  public void sendMessage(String message) {
     rabbitTemplate.convertAndSend("myQueue", message);
  }
  ```

- **Consumer**:@RabbitListener(queues = "myQueue")
  ```
  public void receiveMessage(String message) {
  ```

```
    System.out.println("Received: " + message);
}
```

Apache Kafka Integration

- Add spring-kafka.
- Configure:spring.kafka.bootstrap-servers=localhost:9092

- **Producer**:@Autowired
  ```
  private KafkaTemplate<String, String> kafkaTemplate;

  public void sendMessage(String message) {
    kafkaTemplate.send("myTopic", message);
  }
  ```

- **Consumer**:@KafkaListener(topics = "myTopic")
  ```
  public void consume(String message) {
    System.out.println("Received: " + message);
  }
  ```

Retry and Dead-Letter Queues

- Configure retries:spring.rabbitmq.listener.simple.retry.enabled=true
  spring.rabbitmq.listener.simple.retry.max-attempts=3

19. Spring Boot and Reactive Programming

Introduction to Reactor

- Reactor is a reactive library for non-blocking applications.
- Key classes: Mono (0-1 item), Flux (0-N items).

WebFlux (Reactive REST APIs)

- Add spring-boot-starter-webflux.
- **Example**:@RestController
  ```
  public class ReactiveController {
    @GetMapping("/items")
    public Flux<String> getItems() {
      return Flux.just("Item1", "Item2");
  ```

```
    }
}
```

Reactive MongoDB, Redis

- Add spring-boot-starter-data-mongodb-reactive or spring-boot-starter-data-redis-reactive.
- **Example**:@Repository
  public interface ReactiveUserRepository extends ReactiveCrudRepository<User, String> {
  }

20. Spring Boot with Microservices

Microservices Architecture

- Break applications into small, independent services.
- Each service has its own database and communicates via APIs or messaging.

Inter-Service Communication

- **REST**:@Autowired
  private RestTemplate restTemplate;

  public String callService() {
    return restTemplate.getForObject("http://other-service/api", String.class);
  }

- **Feign**:@FeignClient(name = "other-service")
  public interface OtherServiceClient {
    @GetMapping("/api")
    String getData();
  }

- **Kafka**: Use Kafka for event-driven communication (see Topic 18).

Centralized Logging

- Use ELK Stack (Elasticsearch, Logstash, Kibana) or similar for logging.

Containerization with Docker

- Create a Dockerfile:FROM openjdk:17
  COPY target/myapp.jar app.jar
  ENTRYPOINT ["java", "-jar", "/app.jar"]

CI/CD Pipelines

- Use Jenkins, GitHub Actions, or GitLab CI for automated builds and deployments.

Kubernetes Deployment

- Deploy with Kubernetes:apiVersion: apps/v1
  kind: Deployment
  metadata:
    name: myapp
  spec:
    replicas: 3
    template:
      spec:
        containers:
          - name: myapp
            image: myapp:latest

21. Spring Boot with GraphQL

Introduction to GraphQL

- GraphQL is a query language for APIs, allowing clients to request specific data.

Schema Definition

- Define in schema.graphqls:type Query {
    user(id: ID!): User
  }

  type User {
    id: ID!
    name: String!
  }

Query, Mutation, Subscription

- **Example**:@Controller
  public class UserGraphQLController {
    @QueryMapping
    public User userById(@Argument Long id) {
      return new User(id, "User" + id);
    }
  }

Integration with GraphQL Java Tools

- Add graphql-spring-boot-starter and define the schema.

Expert/Enterprise Level

22. Spring Boot Performance and Tuning

Profiling

- Use tools like VisualVM or YourKit to identify bottlenecks.

Memory and GC Optimization

- Configure JVM options:java -Xms512m -Xmx1024m -jar myapp.jar

Application Tuning

- Optimize database queries and use caching (e.g., Spring Cache with Redis).

Lazy Initialization

- Enable in application.properties:spring.jpa.hibernate.ddl-auto=update
  spring.jpa.properties.hibernate.enable_lazy_load_no_trans=true

23. Spring Boot Security in Production

CSRF/XSS/Clickjacking Protection

- Enable CSRF:http.csrf();

HTTPS and SSL Config

- Configure in application.properties:server.ssl.key-store=classpath:keystore.p12
  server.ssl.key-store-password=password

Securing APIs

- Use API keys or OAuth2 tokens.

Secure Headers and CORS

- Configure CORS:@Bean
  ```
  public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(List.of("http://example.com"));
    UrlBasedCorsConfigurationSource source = new
  UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
  }
  ```

24. Spring Boot with NoSQL

MongoDB

- Add spring-boot-starter-data-mongodb.
- Configure:spring.data.mongodb.uri=mongodb://localhost:27017/mydb

Redis

- Add spring-boot-starter-data-redis.
- Configure:spring.redis.host=localhost
  spring.redis.port=6379

Cassandra

- Add spring-boot-starter-data-cassandra.

Couchbase

- Add spring-boot-starter-data-couchbase.

25. Spring Boot and Event-Driven Architecture

Domain Events

- Define events:public class UserCreatedEvent {
  private final String userId;
  // Constructor, getters
  }

Integration with Kafka/RabbitMQ/EventStore

- Publish events using Kafka or RabbitMQ (see Topic 18).

Event Sourcing Basics

- Store state as a sequence of events.
- Use libraries like EventStoreDB.

26. Internationalization (i18n) and Localization (l10n)

- Configure messages in src/main/resources/messages.properties:greeting=Hello, {0}!

- Use MessageSource:@Autowired
  private MessageSource messageSource;

  public String getGreeting(String name) {
  return messageSource.getMessage("greeting", new Object[]{name}, Locale.US);
  }

27. Custom Starters Development

- Create a custom starter:<dependency>
  <groupId>com.example</groupId>
  <artifactId>my-starter</artifactId>
  <version>1.0.0</version>
  </dependency>

- Define auto-configuration:@Configuration
  @ConditionalOnClass(MyService.class)
  public class MyAutoConfiguration {

```
    @Bean
    public MyService myService() {
        return new MyService();
    }
}
```

28. Spring Boot Native (GraalVM Support)

- Use GraalVM to compile Spring Boot apps to native images.
- Add spring-boot-starter-native.
- Build:mvn spring-boot:build-image

29. Spring Boot in Serverless & Cloud Platforms

AWS Lambda with Spring Boot

- Use spring-cloud-function-adapter-aws.
- Define a function:@Bean
  ```
  public Function<String, String> uppercase() {
      return String::toUpperCase;
  }
  ```

Azure Functions

- Use spring-cloud-function-adapter-azure.

Google Cloud Functions

- Use spring-cloud-function-adapter-gcp.

30. Spring Boot with API Management Tools

Swagger/OpenAPI Integration

- Add springdoc-openapi-ui:<dependency>
  ```
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-ui</artifactId>
  <version>2.0.2</version>
  </dependency>
  ```

- Access Swagger UI at /swagger-ui.html.

API Versioning

- Use @ApiVersion or URL-based versioning:@GetMapping("/v1/users")
  public String getUsersV1() {
    return "Version 1";
  }

API Documentation and Testing

- Use Swagger annotations or Spring REST Docs.

Additional Essential Topics

31. Spring Boot Caching

- Enable caching with @EnableCaching.
- Use @Cacheable, @CachePut, @CacheEvict.
- **Example**:@Cacheable("users")
  public User getUser(Long id) {
    return userRepository.findById(id).orElse(null);
  }

32. Spring Boot Scheduling

- Enable scheduling with @EnableScheduling.
- Use @Scheduled:@Scheduled(fixedRate = 5000)
  public void performTask() {
    System.out.println("Task executed at " + new Date());
  }

33. Spring Boot Email Sending

- Add spring-boot-starter-mail.
- Configure:spring.mail.host=smtp.gmail.com
  spring.mail.username=your-email@gmail.com
  spring.mail.password=your-password

- Send email:@Autowired
  private JavaMailSender mailSender;

```
public void sendEmail(String to, String subject, String body) {
    SimpleMailMessage message = new SimpleMailMessage();
    message.setTo(to);
    message.setSubject(subject);
    message.setText(body);
    mailSender.send(message);
}
```

Conclusion

This comprehensive guide covers everything you need to learn Spring Boot, from beginner to expert levels. Each topic is explained with practical examples, ensuring even absolute beginners can follow along. Practice each example, experiment with the code, and build small projects to solidify your understanding. For further details, refer to the Spring Boot documentation or join the Spring community on forums like Stack Overflow or X.

Happy coding! 🚀